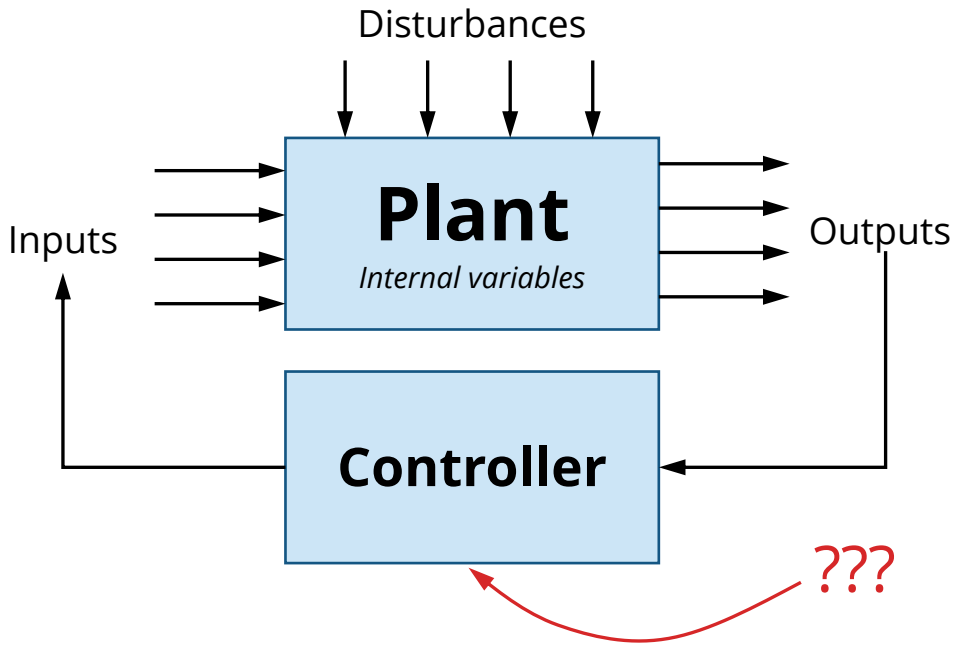


AA 513 Multivariable Control

Lecture 01 // Week 1

Instructor: Prof. Karen Leung

- Name
- Where you are from
- Work/school and interests
- Year in program
- Fun fact: Go-to party trick

Control theory is the ***science of decision-making***. Without control theory, airplanes wouldn't fly, rockets wouldn't launch, and your thermostat wouldn't keep you comfortable.

AI may extend our capabilities, but it is control that forms the **backbone** of how systems operate.

Course goals

Bridging *foundational theory* and *practical application* in multivariable control (and estimation).

Develop skills necessary to formulate control synthesis problems for multivariable dynamical systems.

Build the background knowledge to pursue more advanced topics in decision-making, robotics, AI/ML/RL/DL.

* This course may give you the framework to help you make better decisions in life, or may lead to more decision paralysis...

Course material

- Where to find course material?

  https://uw-ctrl.github.io/lmc-book/

  - Syllabus
  - Lecture materials
  - Exercises

- Where to ask questions?
  - General question useful for the class / anyone could answer: Ed Discussion
  - Private matters for staff only: email ae513-aut25-staff@uw.edu

** Lecture materials are a work-in-progress. Please report any typos/errors/issues. You can email the course staff or make a pull request*

Lecture format

- Lecture first ~2 hours
- Interactive coding in hour 3+ (part of the exercise submissions)
- Office hours / ask questions in hour 4
- Breaks every hour.

- Strongly encouraged to attend lectures synchronously
- Classes be fully remote?

Grading

- **Exercises 30%**
  - Due Weeks 3, 5, 7, 9
  - Weeks 3, 5, 7 due Thursday midnight, Week 9 due Friday midnight
  - Submit on Canvas
  - Self-assessment due 1 week after
- **Research paper presentation 15%**
  - Choose a research paper and present on it to the class
  - Presentations in week 10
- **Tutorial 40%**
  - Pick a topic of your choice
  - Create a ~15 minutes video ”lecture” on the topic
  - Code notebook should accompany the video
- **Peer review of tutorial 15%**
  - Watch a few tutorials
  - Provide feedback, ask questions, answer questions, post comments

Prerequisite knowledge

- MATH MATH MATH. The ”algebra” of
  - Multivariable calculus
  - Linear algebra
- Programming (Python)
  - Using Python
  - Colab / Jupyter notebooks
  - Reading up on documentation
  - Debugging
  - Using genAI to *enhance* your coding skills, **not replace it**

Pre-assessment quiz

- A short quiz to help you assess your math background
- 10 questions
- 15 minutes
- Does not count towards your grade

How did you do?

- Straightforward
  - You will do fine and (hopefully) enjoy this class and get a lot out of it.
- Medium
  - You may need to Google things occasionally, but nothing should be too surprising.
- Difficult
  - You will probably find the course difficult without some self-revision. Being shaky on the math will make coding/debugging challenging.
- **Good news**: Nothing in this course is “top secret.” Almost all theory” is well-established, and easy to find resources online.

After this course, you can understand the “secret sauce” behind…

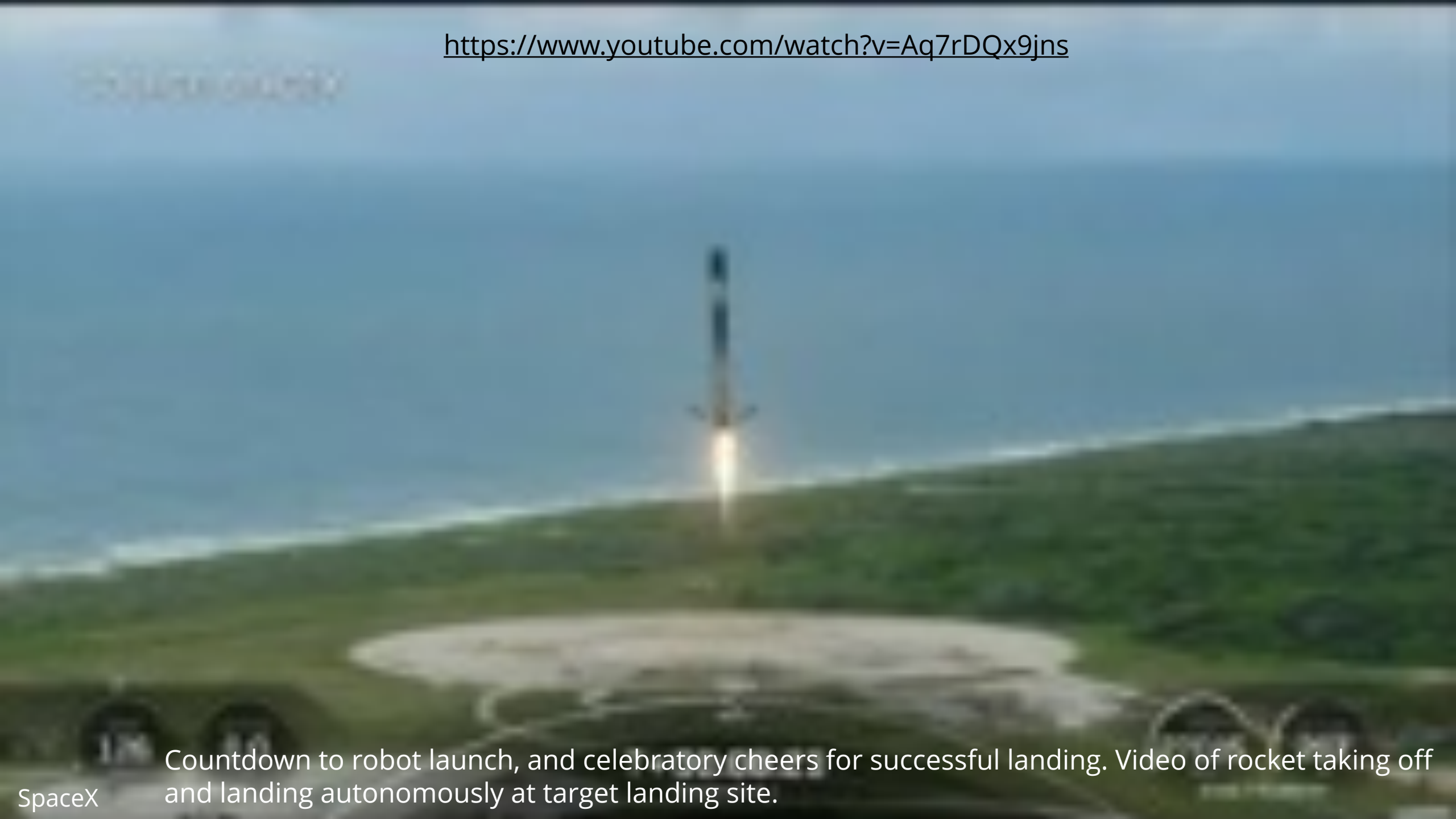
https://www.youtube.com/watch?v=joIsgP9StAY

No voice. Music only. Video of car driving autonomously along a race track without humans inside.

Subosits, J. and Gerdes, J.C. , From the Racetrack to the Road: Real-Time Trajectory Replanning for Autonomous Driving, IEEE Transactions on Intelligent Vehicles, vol. 4, no. 2, 2019

Boston Dynamics No voice. Music only. Humanoid robot dancing to music.

https://www.youtube.com/watch?v=Aq7rDQx9jns



SpaceX

Countdown to robot launch, and celebratory cheers for successful landing. Video of rocket taking off and landing autonomously at target landing site.

What is the secre

MODEL-PREDICTIVE CONTROL

Having identified the boxes, ramps, or barriers in front of the robot and planned a sequence of maneuvers to get over them, the remaining challenge is filling in all of the details needed for the robot to reliably carry out the plan.

Atlas' controller is what's known as a *model-predictive controller* (MPC) because

Exploiting knowledge about the dynamics and applying optimization-based control

VI. L

To transform
quickly, the dyn
tory. While not required to produce a useful modified
the nearer this nominal trajectory is to being feasible
optimal the better since less approximation error wi
introduced by the linearization. The method of discre
chosen to limit the resulting error as much as possibl

A. Linearization

To include the vehicle dynamics as equality cons
quadratic program (QP), we place the equations of mo
following linear form:

$$x(s)' = A(s)x(s) + B(s)u(s),$$

where $x(s) = [\Delta t\ e\ \Delta V\ \sigma]^T$, $u(s) = [\Delta a_x\ \Delta da_y]^T$
notes a perturbation from the nominal value at that p

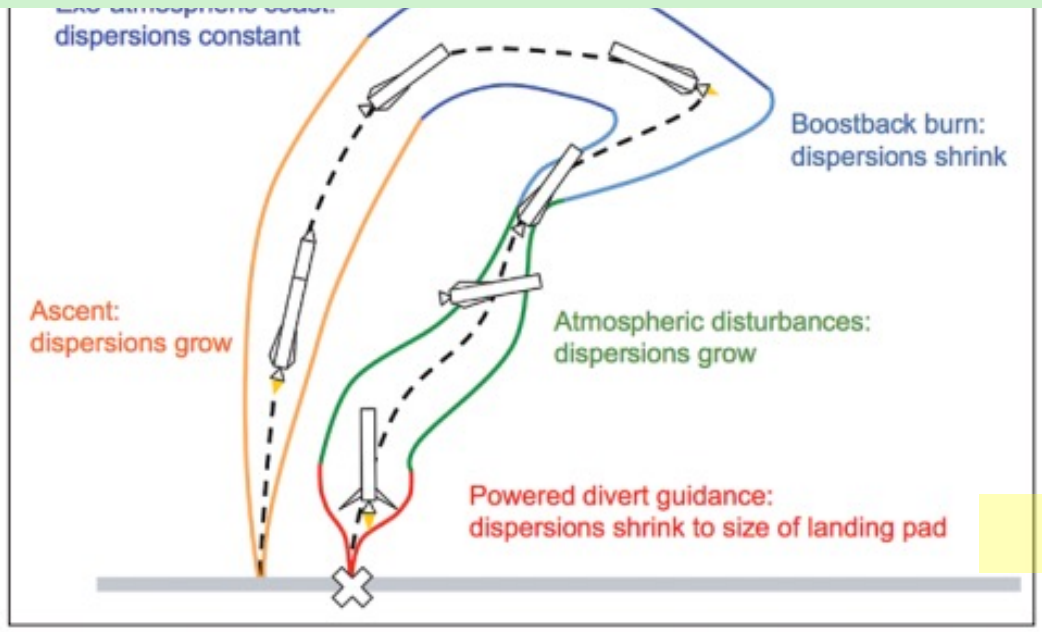


FIGURE 4 Phases of an F9R return-to-launch-site mission. The colored lines represent the largest possible variations in the trajectory, known as dispersions.

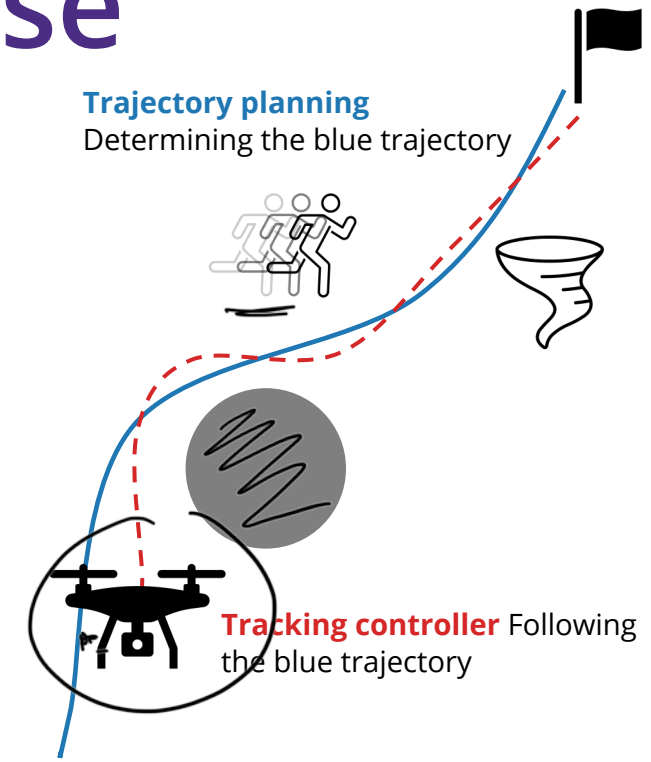
2007;
0).

Because Earth's atmosphere is 100 times as dense as that of Mars, aerodynamic forces become the primary concern rather than a disturbance so small that it can be neglected in the trajectory planning phase. As a result, Earth landing is a very different problem, but SpaceX and Blue Origin have shown that this too can be solved. SpaceX uses CVXGEN (Mattingley and Boyd 2012) to generate customized flight code, which enables very high-speed onboard convex optimization.

Subosits, J. and Gerdes, J.C. , *From the Racetrack to the Road: Real-Time Trajectory Replanning for Autonomous Driving*, IEEE Transactions on Intelligent Vehicles, vol. 4, no. 2, 2019
Boston Dynamics, Atlas, https://www.bostondynamics.com/atlas
SpaceX: https://nap.nationalacademies.org/read/23659/chapter/10

Scope of this course

- A **model-based** approach to control synthesis
  - Assume we have a reasonably good model of the system
- What you can after taking this class
  - Frame a control problem as an optimization problem
  - Make optimization tractable by utilizing properties of the system
- What this course is **not** about
  - Model-free approaches (no model), i.e., reinforcement learning or deep learning
  - Optimization theory
  - State-estimation/filtering course
  - Programming class
- Topics lightly touched but beyond the scope of this course
  - Stochastic control, robust control, nonlinear control, adaptive control, state estimation, reinforcement learning



Potential applications

- Human-robot interactions
- Spacecraft landing/docking
- Aircraft landing/flying
- Delivery robots
- Autonomous driving

(Tentative) Course outline

Week-by-week schedule

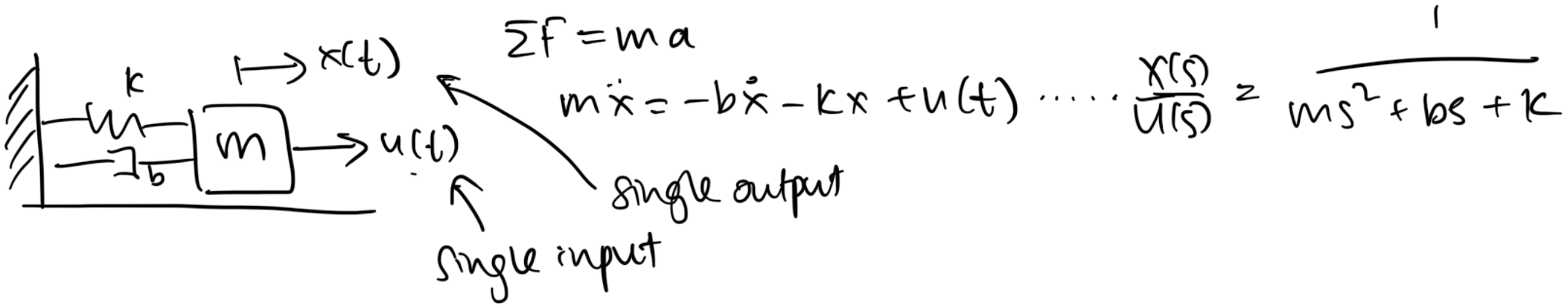
Note: *Italicized* text indicates planned topics, but subject to change.

Date	Week	Topic	Milestones	Materials
Sep 30	1	Introduction, state space representation, coding basics		Ex 1
Oct 7	2	*Optimization, classical feedback control*		
Oct 14	3	*Stability and invariance, Lyapunov and barrier functions*	Week 1-2 exercises due Thurs	
Oct 21	4	*Value function, dynamic programming, Bellman equation*	Self-assessment due Thurs	
Oct 28	5	*Markov Decision Process, POMDPs // Guest lecture: Dr. Liam Kruse*	Week 3-4 exercises due Thurs	
Nov 4	6	*Linear quadratic regulator*	Self-assessment due Thurs	
Nov 11	7	*(No lecture / Veterans Day)*	Week 5-6 exercises due Thurs	
Nov 18	8	*Trajectory optimization, MPC, learning-based control*	Self-assessment due Thurs	
Nov 25	9	*Kalman filter and Linear Quadratic Gaussian*	Week 8-9 exercises due Fri	
Dec 2	10	Research paper presentation	Self-assessment due Fri	
	Finals		Video and feedback due	

Introduction and background

Single-input single-output (SISO)

- Many of you have probably taken a controls course in your UG
  - "Transfer functions", "root locus", frequency response, "Bode plot", "Nyquist", PID, lead-lag/lag-led compensator, stability margin
- Systems with single input/single output



Requirement-based control design

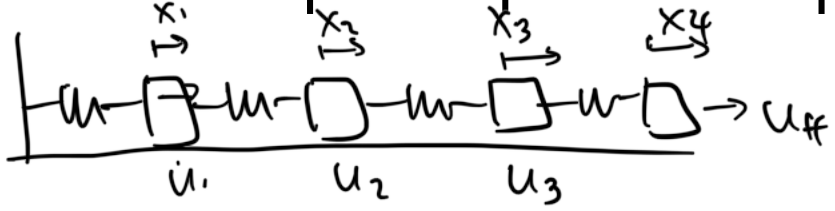
- Controller design can be based on *requirements* or *specifications*
  - E.g., settling time < 3 seconds, steady-state error < ~~0.02~~ 2%, phase margin > K

- there could be multiple controllers that fit the desired requirements.

What if...

- Multiple-input multiple-output?

TF $\frac{\text{output}}{\text{input}}$



- Want to choose the *best* controller?

if K_1 & K_2 meets the requirements, which is better?

- need a criteria, or a metric.

- Respect *constraints* to the system

$u = Ke$ but $|u| \leq u_{max}$

State-space representation

- Need a compact and convenient way to represent and "manipulate" all the relevant variables of the system
- **Definition [State]**

  A smallest subset of system variables that can represent the entire condition (of relevance) of the system at any given time.

  - *Remarks*
    - Not unique
    - Not always observable
    - Appropriate state space representation depends on the level of fidelity desired

$\vec{x} \in \mathbb{R}^n \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = [x_1, \dots x_n]^T \qquad x(t) = \begin{bmatrix} x_1(t) \\ \vdots \\ x_n(t) \end{bmatrix}$

suppose $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ A

is $\tilde{x} = \begin{bmatrix} x_1 \\ x_1 + x_2 \end{bmatrix}$ B

Dynamical systems

- The system states *evolves* over time
- How it evolves depends on the *current* state and time

$\dot{x} = \frac{dx}{dt} = f(x(t), t)$. $f: \mathbb{R}^n \times \mathbb{R} \to \mathbb{R}^n$

$\dot{x} = f(x, t)$, no control input
$\Rightarrow$ autonomous system.

if $\dot{x} = f(x) \Rightarrow$ time-invariant

dynamics are a 1st order ODE

- The dynamics usually come from first-principles
  - Physics for mechanical systems
  - Biology for population dynamics
  - Chemistry for reaction processes

what about $\ddot{x} = -gu$

$z = \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \to \dot{z} = \begin{bmatrix} \dot{x} \\ \ddot{x} \end{bmatrix}$ $\quad \dot{x} = u$

double integrator

Control input

- There could be ways to influence the system dynamics
  - E.g., Rudder, ailerons, elevator, throttle of an aircraft

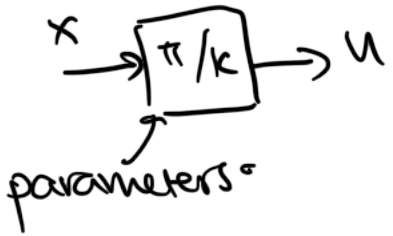
$\vec{u} \in \mathbb{R}^m \quad \vec{u} = \begin{bmatrix} u_1 \\ \vdots \\ u_m \end{bmatrix}$ so our dynamics become $\dot{x} = f(x, u, t)$

"non-autonomous sys".

- We (control engineer) get to choose the value of the control input we get to pick u!

"controller", "policy"

$K(x)$ $\pi(x)$



eg $u = Kx$ (proportional controller)

π/k is a nonlinear function eg. nn.

- output of an algorithm or optimisation problem.

Disturbance input

- Other inputs to the system that we cannot control
  - E.g., Wind on an aircraft, parameter uncertainty

$d \in \mathbb{R}^{m_d}$ $d = \begin{bmatrix} d_1 \\ \vdots \\ d_{m_d} \end{bmatrix}$ $\dot{x} = f(x, u, d, t)$

$\dot{x} = f(x, u, t) + d$

- But we may assume the disturbance input comes from a probability distribution and/or is bounded

$d \sim$ probability dist.

eg $d \sim \mathcal{N}(\mu, \Sigma)$

OR $d \in [d_{min}, d_{max}]$

Observation or measurement

- Typically, do not directly observe the exact value of the state
- Instead, we use observations/measurements from sensors to *estimate* the state
  - E.g., GPS to estimate true position, IMU, pitot tube estimate velocity
- Can use observations from *multiple* sources to perform estimation

$y \in \mathbb{R}^p$, $y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{bmatrix}$ $y = g(x, u, t)$, but actually want $x = g^{-1}(y, u, t)$.

State-space system examples

What is a possible state, control, disturbance, observation? How would you produce dynamics?

- Human body

body as a single mass, look at arms/legs.

- Animal population

population of animals
sample of herd.

- Temperature in a house

Types of dynamics

$$\dot{x} = f(x, u, t)$$

- Nonlinear

$$\dot{x} = f(x, u, t)$$

- Control-affine

$$\dot{x} = f_0(x, t) + B(x, t)u$$

- Linear

no x

$$\dot{x} = A(t)x + B(t)u$$

$$\dot{x} = Ax + Bu$$

- hybrid dynamics
  - some continuous, some discrete.

More examples

- Lorenz attractor

$$\vec{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} \sigma(y - x) \\ x(\rho - z) - y \\ xy - \beta z \end{bmatrix}$$

no u → autonomous sys.

non linear

- Dubins car

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} \quad u = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

non-auto.

$$\begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

control affine

$$\dot{x} = \bigcirc + B(x)u$$

- Dynamic simple car

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \frac{v}{L}\tan\delta \\ a \end{bmatrix} \quad u = \begin{bmatrix} \delta \\ a \end{bmatrix}$$

nonlinear

let $d = \tan\delta$

Discrete-time dynamics

- First-principles typically provide *continuous time* dynamics
- Digital devices operate in *discrete* steps
- Easier to simulate and implement algorithms on digital hardware
- Go from continuous-time dynamics to discrete-time?

$x(t) \rightarrow x_t$, $\dot{x} \rightarrow x_{t+1}$

$\dot{x} = f_c(x, u, t) \longrightarrow x_{t+1} = f_d(x_t, u_t, t)$

$x_{k+1} = f_d(x_k, u_k, t_k)$

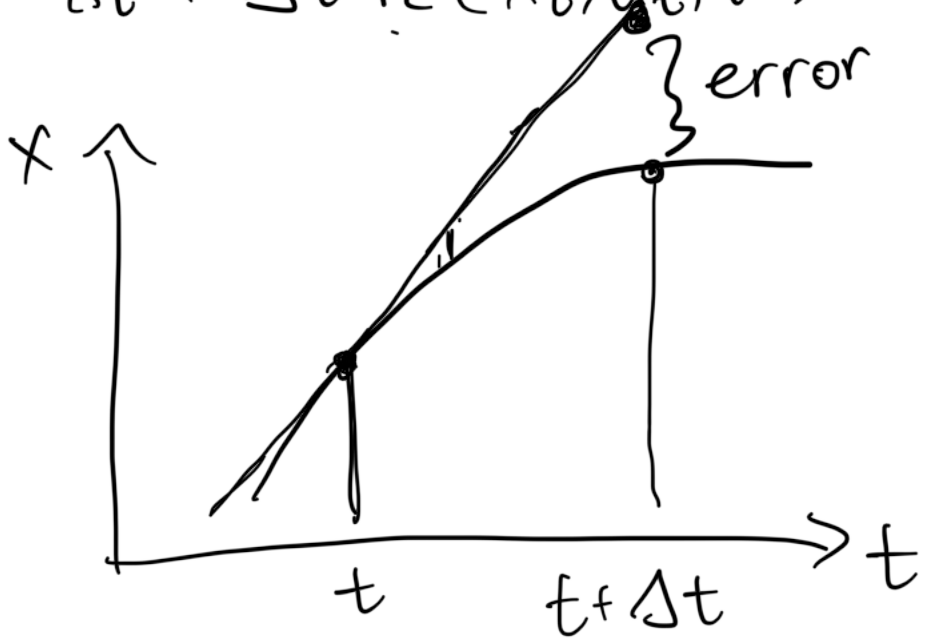
$$x(t+\Delta t) = x(t) + \int_0^{\Delta t} f_c\big(x(t+\tau), u(t+\tau), t+\tau\big)\, d\tau$$

$x(t+\Delta t) \rightarrow x_{t+1}$, $x(t) \rightarrow x_t$

- Performing the integration analytically is not scalable!
- Numerically integrate instead!

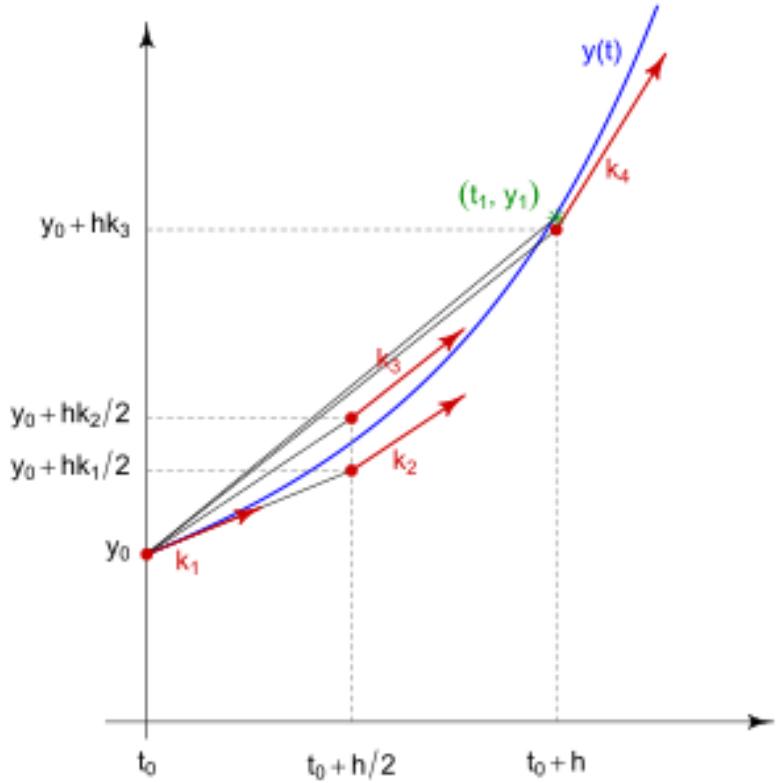
Euler integration

$$x_{t+1} = x_t + \Delta t f_c(x_t, y_t, t)$$



Runge-Kutta integration

$$
\begin{aligned}
k_1 &= f(t_n,\, x_n)\,, \\
k_2 &= f\left(t_n + \tfrac{h}{2},\, x_n + \tfrac{h}{2}k_1\right), \\
k_3 &= f\left(t_n + \tfrac{h}{2},\, x_n + \tfrac{h}{2}k_2\right), \\
k_4 &= f\left(t_n + h,\, x_n + hk_3\right), \\
x_{n+1} &= x_n + \tfrac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).
\end{aligned}
$$

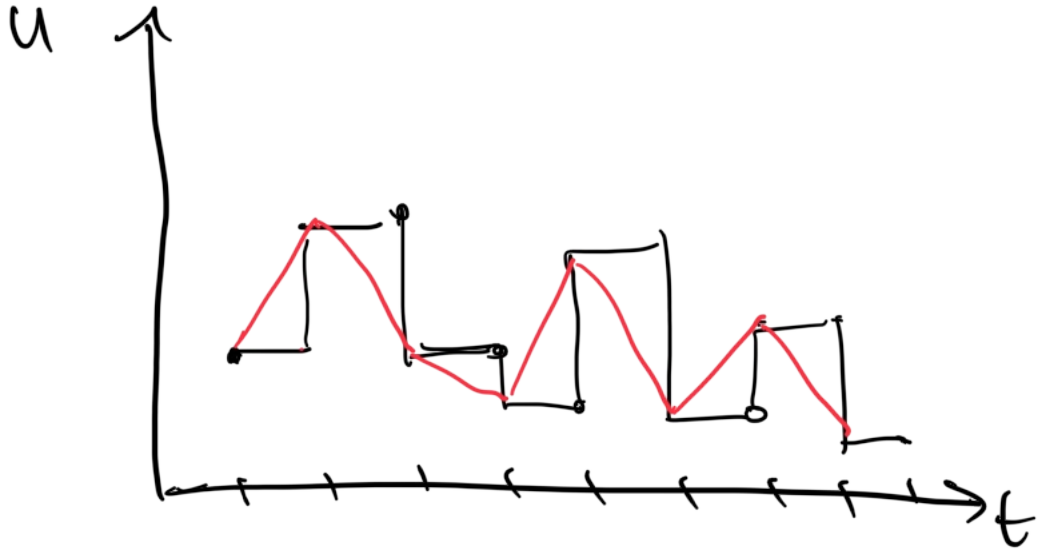


Credit: HilberTraum

Zero-order hold

- What happens to controls (and disturbances) over the integration step?

assume u is constant over Δt



Linear vs nonlinear dynamics

- Linear systems are nice because
  - superposition holds
  - use linear algebra
  - tractable to work with, sometimes even closed form
  - local = global analysis
- Nonlinear systems are **not nice** because
  - not nonlinearities are equal, tools are limited
  - unpredictable behaviours (chaos, bifurcation)
  - no closed form.
  - local ≠ global analysis.

Review: Gradients, Jacobian, Hessians

Linearization

- First-order Taylor approximation

let $f: \mathbb{R}^n \to \mathbb{R}$,

at $x \approx x_0$

$$f(x) \approx f(x_0) + \nabla f(x_0)^T (x - x_0) + \frac{1}{2!} (x - x_0)^T H_f(x_0) (x - x_0) + \dots$$

$f(x_0)$: 0th order

$\nabla f(x_0)^T (x - x_0)$: Jacobian/gradient

0th order + Jacobian/gradient: 1st order

$\frac{1}{2!} (x - x_0)^T H_f(x_0) (x - x_0)$: hessian

hessian: 2nd order

$f: \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$

$(x, u) \approx (x_0, u_0)$

$$f(x, u) \approx f(x_0, u_0) + \nabla_x f(x_0, u_0)^T (x - x_0) + \nabla_u f(x_0, u_0)^T (u - u_0) + \dots$$

$$\approx Ax + Bu + C$$

Interactive coding

- We will be using JAX (https://github.com/jax-ml/jax)

Transformable numerical computing at scale

CI passing | pypi v0.7.2

Transformations | Scaling | Install guide | Change logs | Reference docs

What is JAX?

JAX is a Python library for accelerator-oriented array computation and program transformation, designed for high-performance numerical computing and large-scale machine learning.

JAX can automatically differentiate native Python and NumPy functions. It can differentiate through loops, branches, recursion, and closures, and it can take derivatives of derivatives of derivatives. It supports reverse-mode differentiation (a.k.a. backpropagation) via `jax.grad` as well as forward-mode differentiation, and the two can be composed arbitrarily to any order.

JAX uses XLA to compile and scale your NumPy programs on TPUs, GPUs, and other hardware accelerators. You can compile your own pure functions with `jax.jit`. Compilation and automatic differentiation can be composed arbitrarily.

Dig a little deeper, and you'll see that JAX is really an extensible system for composable function transformations at scale.